

He generado un conjunto completo de **60 pruebas** para UserManagementAPI.http que cubren:

Middleware de Logging

- Todas las peticiones registran duración y status code en Debug.WriteLine

Middleware de Excepciones Globales

- ArgumentNullException (body NULL)
- InvalidOperationException (emails duplicados, lógica de negocio)
- DbUpdateException (errores de BD)
- TimeoutException (si ocurriera)
- Excepciones genéricas (500)

Validación en 3 Capas

- **Handler:** IDs negativos/cero, parámetros NULL, rangos de paginación
- **Validator:** Longitudes, regex (email/nombre), caracteres inválidos, espacios múltiples
- **Service:** Emails duplicados, unicidad en BD

Caché de Memoria

- Pruebas consecutivas para verificar caché (GET users, GET user by ID, búsquedas)
- Invalidación tras operaciones de escritura (POST, PUT, DELETE)

Casos Especiales

- Normalización de emails (lowercase)
- Trim de strings
- Caracteres especiales válidos (acentos, guiones, apóstrofes)
- Paginación con metadatos completos
- Response compression

Las pruebas están numeradas y documentadas con respuestas esperadas para facilitar la verificación sistemática.

```
@UserManagementAPI_HostAddress = http://localhost:5112
```

```
### =====
```

```
### PRUEBAS DE MIDDLEWARE DE LOGGING Y EXCEPCIONES
```

```
### =====
```

```
### 1. GET: Obtener todos los usuarios (paginación válida)
```

```
# Respuesta esperada: 200 OK
```

```
# Middleware: Registra duración y status code
```

```
GET {{UserManagementAPI_HostAddress}}/api/users?page=1&pageSize=10
```

```
Accept: application/json
```

```
###
```

```
### 2. GET: Obtener todos los usuarios (sin parámetros - valores por defecto)
# Respuesta esperada: 200 OK
GET {{UserManagementAPI_HostAddress}}/api/users
Accept: application/json

###

### 3. GET: Paginación inválida (page negativo)
# Respuesta esperada: 400 Bad Request
# Activa validación en handler
GET {{UserManagementAPI_HostAddress}}/api/users?page=0&pageSize=10
Accept: application/json

###

### 4. GET: Paginación inválida (pageSize excede límite)
# Respuesta esperada: 400 Bad Request
# Activa validación de pageSize máximo
GET {{UserManagementAPI_HostAddress}}/api/users?page=1&pageSize=150
Accept: application/json

###

### 5. GET: Obtener usuario por ID válido
# Respuesta esperada: 404 Not Found (DB vacía inicialmente)
GET {{UserManagementAPI_HostAddress}}/api/users/1
Accept: application/json

###

### 6. GET: Obtener usuario por ID inválido (ID negativo)
# Respuesta esperada: 400 Bad Request
GET {{UserManagementAPI_HostAddress}}/api/users/-1
Accept: application/json

###

### 7. GET: Obtener usuario por ID cero
# Respuesta esperada: 400 Bad Request
GET {{UserManagementAPI_HostAddress}}/api/users/0
Accept: application/json

###

### 8. POST: Crear usuario válido
```

```
# Respuesta esperada: 201 Created con Location header
# Middleware: Registra petición POST exitosa
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
  "name": "Juan Pérez",
  "email": "juan.perez@example.com"
}

###

### 9. POST: Crear segundo usuario para pruebas
# Respuesta esperada: 201 Created
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
  "name": "María García",
  "email": "maria.garcia@example.com"
}

###

### 10. POST: Crear usuario con email duplicado
# Respuesta esperada: 400 Bad Request
# Activa InvalidOperationException capturada en handler
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
  "name": "Juan Duplicado",
  "email": "juan.perez@example.com"
}

###

### 11. POST: Crear usuario con nombre vacío
# Respuesta esperada: 400 Bad Request
# Activa validación de validator
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
  "name": "",
  "email": "test@example.com"
}
```

}

###

```
### 12. POST: Crear usuario con nombre muy corto
```

```
# Respuesta esperada: 400 Bad Request
```

```
# Activa validación de longitud mínima (MIN_NAME_LENGTH = 2)
```

```
POST {{UserManagementAPI_HostAddress}}/api/users
```

```
Content-Type: application/json
```

$$\left\{ \begin{array}{l} \text{ } \end{array} \right.$$

}

###

```
### 13. POST: Crear usuario con nombre muy largo
```

```
# Respuesta esperada: 400 Bad Request
```

```
# Activa validación de longitud máxima (MAX_NAME_LENGTH = 100)
```

```
POST {{UserManagementAPI_HostAddress}}/api/users
```

```
Content-Type: application/json
```

{

"NombreMuyLargoQueExcedeLosLimites12345678901234567890123456789012345678901234
567890123456789012345678901234567890123456789012345678901234567890",

}

###

14. POST: Crear usuario con nombre con caracteres inválidos

```
# Respuesta esperada: 400 Bad Request
```

```
# Activa validación de NameRegex
```

```
POST {{UserManagementAPI_HostAddress}}/api/users
```

```
Content-Type: application/json
```

{

}

###

15. POST: Crear usuario con nombre solo números

Respuesta esperada: 400 Bad Request

Activa validación de regex @"^\d+\$"

POST {{UserManagementAPI_HostAddress}}/api/users

Content-Type: application/json

```
{
  "name": "12345",
  "email": "test5@example.com"
}
```

###

16. POST: Crear usuario con nombre con espacios múltiples

Respuesta esperada: 400 Bad Request

Activa validación de espacios múltiples

POST {{UserManagementAPI_HostAddress}}/api/users

Content-Type: application/json

```
{
  "name": "Juan    Pérez",
  "email": "test6@example.com"
}
```

###

17. POST: Crear usuario con email vacío

Respuesta esperada: 400 Bad Request

Activa validación de email vacío

POST {{UserManagementAPI_HostAddress}}/api/users

Content-Type: application/json

```
{
  "name": "Test Usuario",
  "email": ""
}
```

###

18. POST: Crear usuario con email inválido (sin @)

Respuesta esperada: 400 Bad Request

Activa validación de EmailRegex

POST {{UserManagementAPI_HostAddress}}/api/users

Content-Type: application/json

```
{
```

```
"name": "Test Usuario",
"email": "emailinvalido.com"
}

###

### 19. POST: Crear usuario con email inválido (sin dominio)
# Respuesta esperada: 400 Bad Request
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
  "name": "Test Usuario",
  "email": "test@"
}

###

### 20. POST: Crear usuario con email muy corto
# Respuesta esperada: 400 Bad Request
# Activa validación de MIN_EMAIL_LENGTH = 5
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
  "name": "Test Usuario",
  "email": "a@b"
}

###

### 21. POST: Crear usuario con body NULL/vacío
# Respuesta esperada: 400 Bad Request
# Activa validación "datos del usuario son requeridos"
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

###

### 22. POST: Crear tercer usuario para pruebas de actualización
# Respuesta esperada: 201 Created
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
```

```
"name": "Pedro López",
"email": "pedro.lopez@example.com"
}

###

### 23. GET: Verificar usuarios creados con paginación
# Respuesta esperada: 200 OK con lista de usuarios
GET {{UserManagementAPI_HostAddress}}/api/users?page=1&pageSize=5
Accept: application/json

###

### 24. PUT: Actualizar usuario existente (cambiar nombre)
# Respuesta esperada: 200 OK
# Nota: Ajustar ID según usuarios creados (normalmente ID=1)
PUT {{UserManagementAPI_HostAddress}}/api/users/1
Content-Type: application/json

{
  "name": "Juan Carlos Pérez Actualizado"
}

###

### 25. PUT: Actualizar usuario existente (cambiar email)
# Respuesta esperada: 200 OK
PUT {{UserManagementAPI_HostAddress}}/api/users/1
Content-Type: application/json

{
  "email": "juan.carlos.nuevo@example.com"
}

###

### 26. PUT: Actualizar usuario existente (cambiar nombre y email)
# Respuesta esperada: 200 OK
PUT {{UserManagementAPI_HostAddress}}/api/users/2
Content-Type: application/json

{
  "name": "María Fernanda García",
  "email": "maria.fernanda@example.com"
}
```

###

27. PUT: Actualizar usuario con ID inválido (negativo)

Respuesta esperada: 400 Bad Request

PUT {{UserManagementAPI_HostAddress}}/api/users/-5

Content-Type: application/json

```
{  
  "name": "Test"  
}
```

###

28. PUT: Actualizar usuario inexistente

Respuesta esperada: 404 Not Found

PUT {{UserManagementAPI_HostAddress}}/api/users/999

Content-Type: application/json

```
{  
  "name": "Usuario Inexistente"  
}
```

###

29. PUT: Actualizar usuario sin proporcionar campos

Respuesta esperada: 400 Bad Request

Activa validación "Debe proporcionar al menos un campo"

PUT {{UserManagementAPI_HostAddress}}/api/users/1

Content-Type: application/json

```
{  
}
```

###

30. PUT: Actualizar usuario con email duplicado

Respuesta esperada: 400 Bad Request

Activa InvalidOperationException de email duplicado

PUT {{UserManagementAPI_HostAddress}}/api/users/1

Content-Type: application/json

```
{  
  "email": "maria.fernanda@example.com"  
}
```


###

31. PUT: Actualizar usuario con nombre inválido

Respuesta esperada: 400 Bad Request

PUT {{UserManagementAPI_HostAddress}}/api/users/1

Content-Type: application/json

```
{  
  "name": "X"  
}
```

###

32. PUT: Actualizar usuario con email inválido

Respuesta esperada: 400 Bad Request

PUT {{UserManagementAPI_HostAddress}}/api/users/1

Content-Type: application/json

```
{  
  "email": "email-invalida"  
}
```

###

33. GET: Buscar usuarios por nombre (query válido)

Respuesta esperada: 200 OK con resultados

GET {{UserManagementAPI_HostAddress}}/api/users/search/Juan

Accept: application/json

###

34. GET: Buscar usuarios por email parcial

Respuesta esperada: 200 OK con resultados

GET {{UserManagementAPI_HostAddress}}/api/users/search/example.com

Accept: application/json

###

35. GET: Buscar usuarios con query muy corto

Respuesta esperada: 400 Bad Request

Activa validación "búsqueda debe tener al menos 2 caracteres"

GET {{UserManagementAPI_HostAddress}}/api/users/search/a

Accept: application/json

###

36. GET: Buscar usuarios con query vacío

Respuesta esperada: 400 Bad Request

GET {{UserManagementAPI_HostAddress}}/api/users/search/

Accept: application/json

###

37. GET: Buscar usuarios sin resultados

Respuesta esperada: 200 OK con lista vacía

GET {{UserManagementAPI_HostAddress}}/api/users/search/NoExiste123

Accept: application/json

###

38. GET: Probar caché de búsqueda (segunda llamada idéntica)

Respuesta esperada: 200 OK (debería ser más rápida, viene de caché)

Middleware muestra duración menor

GET {{UserManagementAPI_HostAddress}}/api/users/search/Juan

Accept: application/json

###

39. GET: Probar caché de paginación

Respuesta esperada: 200 OK (segunda llamada con mismos parámetros)

GET {{UserManagementAPI_HostAddress}}/api/users?page=1&pageSize=5

Accept: application/json

###

40. GET: Obtener usuario específico para probar caché individual

Respuesta esperada: 200 OK

GET {{UserManagementAPI_HostAddress}}/api/users/1

Accept: application/json

###

41. GET: Segunda llamada al mismo usuario (debería venir de caché)

Respuesta esperada: 200 OK (más rápida)

GET {{UserManagementAPI_HostAddress}}/api/users/1

Accept: application/json

###

```
### 42. DELETE: Eliminar usuario existente
# Respuesta esperada: 204 No Content
# Invalida caché de todos los listados
DELETE {{UserManagementAPI_HostAddress}}/api/users/3
Accept: application/json

###

### 43. DELETE: Eliminar usuario con ID inválido (negativo)
# Respuesta esperada: 400 Bad Request
DELETE {{UserManagementAPI_HostAddress}}/api/users/-1
Accept: application/json

###

### 44. DELETE: Eliminar usuario inexistente
# Respuesta esperada: 404 Not Found
DELETE {{UserManagementAPI_HostAddress}}/api/users/999
Accept: application/json

###

### 45. DELETE: Intentar eliminar usuario ya eliminado
# Respuesta esperada: 404 Not Found
DELETE {{UserManagementAPI_HostAddress}}/api/users/3
Accept: application/json

###

### 46. GET: Verificar que el usuario fue eliminado
# Respuesta esperada: 404 Not Found
GET {{UserManagementAPI_HostAddress}}/api/users/3
Accept: application/json

###

### 47. GET: Verificar lista actualizada tras eliminación
# Respuesta esperada: 200 OK (caché invalidado, nueva consulta a BD)
GET {{UserManagementAPI_HostAddress}}/api/users?page=1&pageSize=10
Accept: application/json

###

### 48. POST: Crear usuario con caracteres especiales válidos en nombre
# Respuesta esperada: 201 Created
```

```
# Permite: a-zA-Z0-9\s\-'áéíóúñ
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
  "name": "José-María O'Connor",
  "email": "jose.maria@example.com"
}

###

### 49. POST: Crear usuario con acentos en nombre
# Respuesta esperada: 201 Created
POST {{UserManagementAPI_HostAddress}}/api/users
Content-Type: application/json

{
  "name": "Sofía Ñoño",
  "email": "sofia.nono@example.com"
}

###

### 50. GET: Verificar paginación con segunda página
# Respuesta esperada: 200 OK
GET {{UserManagementAPI_HostAddress}}/api/users?page=2&pageSize=2
Accept: application/json

###

### 51. GET: Verificar metadatos de paginación (HasNextPage, HasPreviousPage)
# Respuesta esperada: 200 OK con metadatos correctos
GET {{UserManagementAPI_HostAddress}}/api/users?page=1&pageSize=2
Accept: application/json

###

### 52. GET: Página fuera de rango (debería devolver datos vacíos)
# Respuesta esperada: 200 OK con Data vacía
GET {{UserManagementAPI_HostAddress}}/api/users?page=100&pageSize=10
Accept: application/json

###

### =====
```

PRUEBAS ADICIONALES DE CASOS EXTREMOS

=====

53. POST: Email con mayúsculas (debería normalizarse a lowercase)

Respuesta esperada: 201 Created

POST {{UserManagementAPI_HostAddress}}/api/users

Content-Type: application/json

```
{
  "name": "Test Mayusculas",
  "email": "TEST.MAYUSCULAS@EXAMPLE.COM"
}
```

###

54. GET: Buscar por email en mayúsculas (debería encontrar)

Respuesta esperada: 200 OK con resultados

GET {{UserManagementAPI_HostAddress}}/api/users/search/MAYUSCULAS

Accept: application/json

###

55. POST: Nombre con espacios al inicio/final (debería aplicar Trim)

Respuesta esperada: 201 Created

POST {{UserManagementAPI_HostAddress}}/api/users

Content-Type: application/json

```
{
  "name": "  Espacios Test  ",
  "email": "espacios.test@example.com"
}
```

###

56. GET: Verificar todos los usuarios finales

Respuesta esperada: 200 OK con listado completo

GET {{UserManagementAPI_HostAddress}}/api/users?page=1&pageSize=20

Accept: application/json

###

57. GET: Endpoint inexistente (activa middleware de excepciones)

Respuesta esperada: 404 Not Found

GET {{UserManagementAPI_HostAddress}}/api/usuarios-no-existe

Accept: application/json

###

58. POST: Múltiples errores de validación simultáneos
Respuesta esperada: 400 Bad Request con array de errores

POST {{UserManagementAPI_HostAddress}}/api/users

Content-Type: application/json

```
{  
  "name": "A",  
  "email": "email-invalido"  
}
```

###

59. PUT: Body NULL en actualización

Respuesta esperada: 400 Bad Request

PUT {{UserManagementAPI_HostAddress}}/api/users/1

Content-Type: application/json

###

60. GET: Verificar compresión HTTPS (Response Compression)

Respuesta esperada: 200 OK con header Content-Encoding

GET {{UserManagementAPI_HostAddress}}/api/users

Accept: application/json

Accept-Encoding: gzip, deflate, br

###